

MILESTONE 1: DATASET CREATION

Davide Falessi

MILESTONE 1

- Each student must work on **one project** as identified via this algorithm:

1. Take the first letter of your last name (e.g., F for Falessi)
2. Take the number of the letter (e.g., F = 6)
3. $X = \text{number} \bmod 6$ (e.g. $X = 0$)

switch (X) {

case 0: project == AVRO;

case 1: project == OPENJPA;

case 2: project == STORM;

case 3: project == ZOOKEEPER;

case 4: project == SYNCOPE;

case 5: project == TAJO;

}

MILESTONE 1

- For your project create a table (CSV file) having the following columns:
 - Name of the project: AVRO
 - Name of the class: /ddd/sss/dddd/ccc/caption.java
 - Release ID
 - About 20 features + NSmells
 - Bugginess (yes/no)

CLASS METRICS

- **Size (LOC)*:** lines of code.
- **LOC Touched*:** sum over revisions of LOC added and deleted.
- **NR*:** number of revisions.
- **Nfix*:** number of defect fixes.
- **Nauth*:** number of authors.
- **LOC Added*:** sum over revisions of LOC added.
- **Max LOC Added*:** maximum over revisions of LOC added.
- **Average LOC Added*:** average LOC added per revision.
- **Churn*:** sum over revisions of added and deleted LOC.
- **Max Churn*:** maximum churn over revisions.
- **Average Churn*:** average churn over revisions.
- **Change Set Size*:** number of files committed together.
- **Max Change Set*:** maximum change set size over revisions.
- **Average Change Set*:** average change set size over revisions.
- **Age:** age of release.
- **Weighted Age:** age of release weighted by LOC touched.

- *= within the release or from release 0.
- See example

COMMIT METRICS

- **Number of modified subsystems (NS):** changes modifying many subsystems are more likely to be defect-prone.
- **Number of modified directories (ND):** changes that modify many directories are more likely to be defect-prone.
- **Number of modified files (NF):** changes touching many files are more likely to be defect prone.
- **Distribution of modified code across each file (Entropy):** changes with high entropy are more likely to be defect-prone, because a developer will have to recall and track higher numbers of scattered changes across each file.
- **Lines of code added (LA):** the more lines of code added the more likely a defect is introduced.
- **Lines of code deleted (LD):** the more lines of code deleted the higher the chance of a defect to occur.
- **Lines of code in a file before the change (LT):** the larger a file the more likely a change might introduce a defect.
- **Whether or not the change is a defect fix (FIX):** fixing a defect means that an error was made in an earlier implementation, therefore it may indicate an area where errors are more likely.
- **Number of developers that changed the modified files (NDEV):** the larger the NDEV, the more likely a defect is introduced because files revised by many developers often contain different thoughts and coding styles.
- **Average time interval between the last and the current change (AGE):** the lower the AGE, the more likely a defect will be introduced.
- **Number of unique changes to the modified files (NUC):** the larger the NUC, the more likely a defect is introduced, because a developer will have to recall and track many previous changes.
- **Developer experience (EXP):** more experienced developers are less likely to introduce a defect.
- **Recent developer experience (REXP):** developers that have often modified the files in recent months are less likely to introduce a defect because they will be more familiar with the recent developments in the system.
- **Developer experience on a subsystem (SEXP):** developers that are familiar with the subsystems modified by a change are less likely to introduce a defect.

POSSIBLE WORKFLOW

1. Given proj X
2. Find releases and related dates
3. Ignore last 66% of releases
4. For each release*
 1. Find the last commit of this release
 2. Check out the (entire) code related to that revision
 3. For each java class
 1. Find the name
 2. Compute the features
 3. Compute Nsmells via SonarCloud or PMD or similar
 4. Write new line in CSV
5. **Labeling**
 - Assume all classes to be not buggy
 - Find all tickets that are: Type == “defect” AND (status == “Closed” OR status == “Resolved”) AND Resolution == “Fixed”.
 - Apply Proportion Total: i.e., use SZZ to identify buggy classes, use affected (or fixed version) if available, otherwise compute and use P on all tickets*, to identify the set of releases where the classes should be labeled as buggy.

PROVIDED JAVA CODE

- Release identification
- Ticket identification